

Boosting with Lexicographic Programming: Addressing Class Imbalance without Cost Tuning

Shounak Datta^{ID}, Sayak Nag^{ID}, and Swagatam Das^{ID}

Abstract—A large amount of research effort has been dedicated to adapting boosting for imbalanced classification. However, boosting methods are yet to be satisfactorily immune to class imbalance, especially for multi-class problems. This is because most of the existing solutions for handling class imbalance rely on expensive cost set tuning for determining the proper level of compensation. We show that the assignment of weights to the component classifiers of a boosted ensemble can be thought of as a game of *Tug of War* between the classes in the margin space. We then demonstrate how this insight can be used to attain a good compromise between the rare and abundant classes without having to resort to cost set tuning, which has long been the norm for imbalanced classification. The solution is based on a lexicographic linear programming framework which requires two stages. Initially, class-specific component weight combinations are found so as to minimize a hinge loss individually for each of the classes. Subsequently, the final component weights are assigned so that the maximum deviation from the class-specific minimum loss values (obtained in the previous stage) is minimized. Hence, the proposal is not only restricted to two-class situations, but is also readily applicable to multi-class problems. Additionally, we also derive the dual formulation corresponding to the proposed framework. Experiments conducted on artificial and real-world imbalanced datasets as well as on challenging applications such as hyperspectral image classification and ImageNet classification establish the efficacy of the proposal.

Index Terms—Boosting, imbalanced classification, lexicographic linear programming, cost set tuning, multi-criterion decision making

1 INTRODUCTION

BOOSTING [1] is an ensemble learning technique that operates by repeatedly training a so-called weak classifier on reweighted versions of the basic dataset. The reweighting is done so that data instances misclassified in the previous round are assigned greater weights in the current round. The variants thus trained become the component classifiers of the ensemble having weightage proportional to their performance on the training data. Boosting is known to exhibit resistance to overfitting for noise-free datasets, owing to its ability to optimize the margin of the underlying weighted combination of weak classifiers [2].

However, boosting methods (and classification techniques in general) are unable to properly handle datasets characterized by class imbalance of data, i.e., when one or more (but not all) classes in the dataset have a small number of representatives in the training sample. Such imbalanced or uneven datasets often arise in critical real life applications such as medical diagnosis [3], fraud detection [4], etc. In fact, even the state-of-the-art deep-learning techniques which are used for complex computer vision applications, suffer due to class imbalance [5]. Nikolaou et al. [6], in a rather exhaustive comparative study, observe that a

significant amount of research effort has been aimed towards adapting boosting methods like AdaBoost [7] for such class imbalanced learning tasks. Despite continued research efforts [8], [9], [10], [11], [12], [13], [14], [15], [16], classification methods in general and boosting methods in particular have yet to become sufficiently immune to class imbalance due to the following reasons:

- Most of the boosting variants proposed to handle class imbalance [6], [8], [11], [17] assume that the relative costs of misclassifying the two classes are known *a priori*. This is often not true and the set of relative costs that are most suitable for a particular dataset must be found by a costly parameter tuning regime.
- Moreover, most of the research efforts have been aimed at handling dichotomous or the so-called two-class imbalanced problems, with little attention being accorded to multi-class or polychotomous classification problems characterized by class imbalance. One of the reasons behind this is the need for parameter tuning, which becomes costlier for multi-class datasets. This is because, in a multi-class situation, either an entire matrix of costs must be estimated [18], or multiple relative costs (pertaining to multiple two-class problems obtained by decomposing the multi-class problem) must be tuned [19].
- Moreover, imbalanced classification on multi-class datasets is further complicated by the fact that the class imbalance can be of multi-minority type (the case where multiple classes are underrepresented), multi-majority type (the case where only a single

- S. Datta and S. Das are with the Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, West Bengal 700108, India. E-mail: shounak.jaduniv@gmail.com, swagatam.das@isical.ac.in.
- S. Nag is with the DST Center of Excellence, Center of Atmospheric Sciences, Indian Institute of Technology, Delhi, India. E-mail: sayak.nag9@gmail.com.

Manuscript received 14 Feb. 2018; revised 11 Nov. 2018; accepted 12 Jan. 2019. Date of publication 22 Jan. 2019; date of current version 1 Apr. 2020.
(Corresponding author: Swagatam Das.)

Recommended for acceptance by L. B. Holder.

Digital Object Identifier no. 10.1109/TKDE.2019.2894148

class has significantly lower number of representatives compared to other classes), or a combination thereof [20].

1.1 Literature on Boosting Techniques Tailored for Handling Class Imbalance

Several improvements over traditional boosting techniques have been proposed to tackle the problem of class imbalance. Some of the approaches integrate data balancing techniques with boosting [13]. For example, SMOTEBoost [21], RAMOBoost [22], DataBoost-IM [23], and AMDO [24] are amalgamations of minority class oversampling and boosting methods. On the other hand, JOUS-Boost [25] and RUSBoost [26] combine boosting with majority class undersampling. Wang and Yao [20] combined Negative Correlation based AdaBoost (AdaBoost.NC) with random oversampling of the minority classes to handle multi-class imbalanced problems.

There also exist methods which combine cost-sensitive learning (i.e., higher weight is assigned to the minority class) with boosting. The solutions range across techniques which incorporate cost-sensitivity into the update rule for the point-wise weights [8], [9], methods which modify the schemes for both point-wise weight update and component classifiers weights [10], [27], and techniques which induce cost-sensitivity into the error function and modify all aspects of boosting to comply with the modified error function [11], [12].

Another approach is to calibrate the ensemble learned by AdaBoost, post-training, to make the output scores of the ensemble correspond with class-probability estimates. Thereafter, the expected misclassification cost can be minimized by selecting the optimal threshold on the calibrated classifier scores, when the costs of misclassification for both the classes are known for a two-class problem. Nikolaou and Brown [17] showed that a key advantage of this method, AdaBoost with Minimum Expected Cost and Calibration (AdaMEC-Calib), is the ability to account for changes in class imbalance without having to retrain the ensemble.

Based on the generally accepted notion that the minimum margin is a key to the generalization performance, Grove and Schuurmans [28] presented an interesting variant of boosting, called LPAdaBoost, where the weights of the component classifiers obtained by AdaBoost are chosen by a Linear Program (LP) so as to maximize the minimum margin (margin can be thought of as the distance from the decision boundary in the direction of proper classification; see Section 1.2 for a formal definition). While LPAdaBoost is not aimed at handling class imbalance, it has innate ability to mitigate the effects of class imbalance. Since the component classifiers generated by AdaBoost are likely to be overwhelmed by the abundance of the majority class instances, most of the data points would tend to be placed into the majority class. This would result in high margins for the majority instances and low margins for the minority instances. Therefore, the minimum margin is likely to correspond to a minority instance. Consequently, LPAdaBoost, in its attempt to increase the minimum margin (corresponding to the most difficult minority point), would also improve the performance on the rest of the minority instances. However, in the presence of outliers, data noise or label noise, the

minimum margin is likely to correspond to corrupt instances, leading to a complete miscalibration of the component weights. Leskovec and Shawe-Taylor [29] proposed LPU-Boost, attempting to solve this problem by allowing regularization of the outliers and assigning higher regularization cost for the minority instances.

It is important to understand that the performance of most of these boosting variants which are capable of tackling class imbalance require expensive cost set tuning to achieve optimal performance. For example, the proper extent of oversampling or undersampling for the sampling based techniques and the appropriate set of relative costs for the cost-sensitive techniques need to be determined by cost tuning. The best set of relative weights depends on a variety of factors such as the relative densities of the classes, the extent and structure of the overlap (if any) between the classes, the extent of noise, the number of outliers, etc. [30].

1.2 Boosting as a Game of Tug of War

A clear understanding of the effects of the component weights on the margin values can help us devise a cost-independent boosting method for handling class imbalance. Towards this end, we begin by formally defining margin in the context of boosting ensembles.

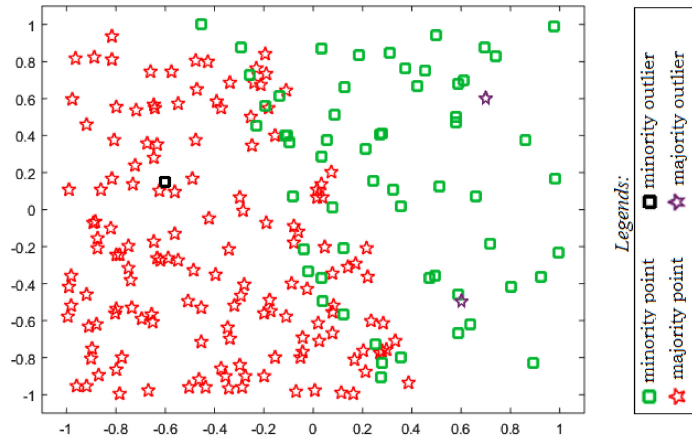
Definition 1. Let $X = \{(\mathbf{x}_i, y_i) : i = 1, 2, \dots, n; y_i \in \{-1, +1\}\}$ be a given training dataset and let $\{f_1, f_2, \dots, f_T\}$ be the set of component classifiers having corresponding set of component weights $\{\alpha_1, \alpha_2, \dots, \alpha_T\}$. Then for a data point $\mathbf{x}_i \in X$, the margin $\rho(\mathbf{x}_i)$ is defined as

$$\rho(\mathbf{x}_i) = y_i h(\mathbf{x}_i),$$

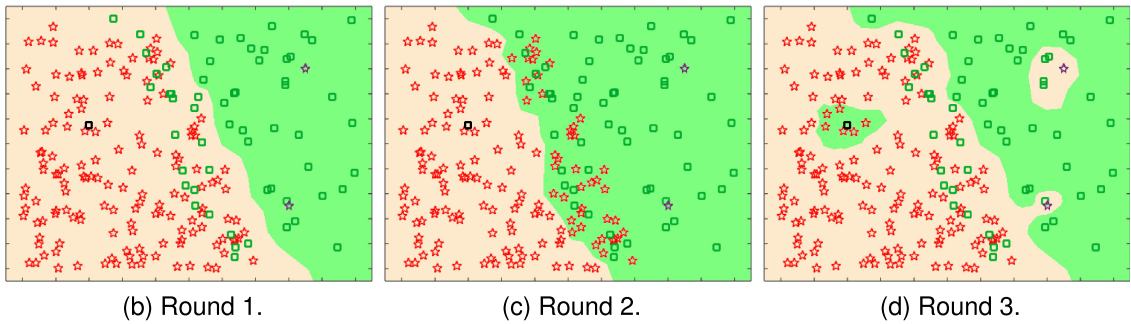
$$\text{where } h(\mathbf{x}_i) = \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i),$$

and is hereafter referred as the “signed margin” of the point $\mathbf{x}_i$ while $f_t(\mathbf{x}_i)$ denotes the output of the t th classifier for $\mathbf{x}_i$.

Let us consider the simple imbalanced dataset exhibited in Fig. 1a. It is seen that both the majority as well as the minority classes contain outliers. We run three rounds of AdaBoost on this dataset to obtain three component classifiers (see Figs. 1b, 1c, and 1d). Ideally, if the component classifiers output labels in the set $\{-1, +1\}$, each point in the positive class should have a signed margin value of +1 while all the negative points should have a signed margin value of -1 (i.e., all points should ideally have margin values of ± 1). An inspection of Figs. 1b, 1c, and 1d shows that some of the points from both the classes are always correctly classified. These points will always attain the ideal margin value, irrespective of the component weights. However, the margin values attained by other points, which are correctly classified only by some of the components, depends on the choice of component weights. If the components which correctly classify most of the majority class instances (generally at the cost of the minority class instances) are assigned high weightage, most of the majority points will have high margin values while many of the minority points will have low margin values. Similarly, if high weightage is assigned to classifiers performing well on the minority points, most minority points will have high margin values at the expense



(a) Toy class imbalanced dataset with outliers in both classes.



(b) Round 1.

(c) Round 2.

(d) Round 3.

Fig. 1. Toy class imbalanced dataset along with the classifiers trainer in three rounds of boosting.

of having low margin values for the majority instances. Therefore, one can think of the problem of component weight assignment as a game of *Tug of War*¹ in the margin space (in the sense increase in the margin values for minority points results in decrease in the margin values for majority points, and *vice versa*).

1.3 Tug of War for Handling Class Imbalance

Intuitively, if both the classes are of equal importance or if no cost information is available, the best trade-off solution is to select the component weights so that similar fractions of data points are misclassified from both classes. A common way to quantify the extent of misclassification from the classes, employed by methods like LPBoost [33] and LPUBoost [29], is to measure the average hinge losses (on the difference between the actual and ideal margin values) for the classes.

Definition 2. For a data instance $\mathbf{x}_i \in X$, the hinge loss on the difference $\rho(\mathbf{x}_i) - 1$ is defined as

$$L_h(\mathbf{x}_i) = \begin{cases} 0 & \text{if } \rho(\mathbf{x}_i) \geq 1, \\ 1 - \rho(\mathbf{x}_i) & \text{if } \rho(\mathbf{x}_i) < 1. \end{cases}$$

1. *Tug of War* is formally defined in mathematics as a zero-sum two-player game where the losses for the two players add up to zero [31], [32]. Our formulation is analogous to these formulations in the sense that the two classes can be considered as the two players and increase in the margin values for points in one class must come at the expense of decrease in the margin values for points from the opposite class. Yet, our formulation is distinct in that the net sum of changes in the margin values may not be zero.

In the absence of outliers, similar average hinge losses from the two classes is likely to indicate similar fractions of misclassification. However, in the presence of outliers (due to noise or otherwise), the high hinge losses for the outlier instances can result in disproportionate increase in the average hinge losses, resulting in miscalibration. Such miscalibration leads to the failure of hard margin maximization [33] in the presence of outliers. Therefore, there is a need to regularize the outliers by some means. Therefore, we are motivated to devise a new framework, based on the *Tug of War* analogy, which can effectively regularize the outlier instances. Consequently, this framework can be used to strike a good compromise between the two classes (in the sense of having similar fractions of misclassification of non-outlier instances), without resorting to cost set tuning.

The *Tug of War* game for the example presented in Fig. 1 is illustrated in Fig. 2. The solutions designated as A, and G in Fig. 2 respectively correspond to the cases where all points from the minority and majority classes attain the ideal margin value. In real-world applications, some difficult instances may be misclassified by all component classifiers. Hence, such ideal solutions are often unattainable in practice and may not correspond to any set of feasible component weights. The solutions B, and F correspond to the sets of component weights which minimize the individual average hinge losses respectively for the minority and majority classes. Being surrounded by points from the opposite class, outliers are generally correctly classified only after accumulating high weightage. However, at such high weights, the component classifiers are likely to misclassify many of the non-outlier points due to the influence of the borderline and/or outlier

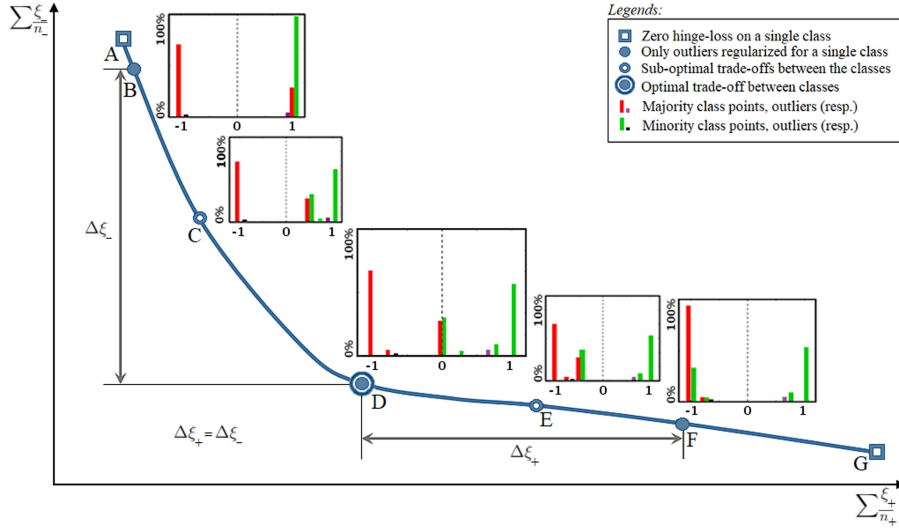


Fig. 2. *Tug of War* between the classes: Solutions A and G, respectively, correspond to ideal performance on the minority and majority classes, and may not be attainable in practice on most datasets. B and F correspond to the respective best attainable performances with only the outliers having non-ideal signed margins. C, D, and E are trade-off solutions with non-outlier points from both classes having non-ideal signed margins. Solution D yields the optimal trade-off with similar fractions of non-outlier points having zero margin (i.e., prone to being misclassified) from both classes.

instances from the opposite class. Consequently, these components are generally assigned lower weightage while attempting to minimize the average hinge loss, resulting in the regularization of the outliers (in the sense that the outliers have worse signed margin and consequently higher loss). Now, any set of component weights which seeks to achieve a balance between the classes will result in an increase beyond the individual minimum average hinge losses for both the classes. Solutions C, D, and E in Fig. 2 correspond to such component weight combinations. Since the outliers already had high loss values, most of this increase in loss will be due to the misclassification of the non-outlier instances. Therefore, similar increase in the average hinge losses of the two classes will correspond to similar fraction of misclassification of non-outlier points for the two classes. Hence, the optimal trade-off between the classes can be achieved by finding the set of components weights corresponding to equal increase in the average hinge loss values for the minority and majority classes, w. r. t. the solutions B and F, respectively. Solution D in Fig. 2 corresponds to the optimal trade-off characterized by similar fractions of non-outlier points from the two classes having margin values close to zero.

1.4 Brief Overview of Literature on Lexicographic Programming

In this paper, we show how the optimal trade-off among classes can be achieved by using Lexicographic Programming (LxP). LxP is concerned with solving a hierarchy of optimization problems where the objective function and/or the constraints imposed on a problem in the hierarchy depend on the optimal solutions obtained for one or more of the problems solved in prior stages of the hierarchy. A special case of LxP consists of solving a sequence (i.e., a hierarchy with exactly one problem in each stage) of optimization problems. This framework is generally employed for Multi-Criteria Decision Making (MCDM) in the forms of Lexicographic Goal Programming (LGP) and Lexicographic Multi-Objective Programming (LMOP) [34], [35]. LGP attempts to attain predefined goals for a set of

objectives which are arranged in decreasing order of priority. LMOP, on the other hand, aims to minimize the set of objectives in order of priority. Due to ease of solving LP problems, LGP and LMOP problems are traditionally formulated as a sequence of LPs [34], [36], [37]. There have also been efforts to reduce LGP and LMOP problems to single objective optimization problems using various approaches [35], [38], [39]. However, such reductions are generally not applicable to the general LxP problems consisting of multiple optimization problems in each stage of the hierarchy. Romero [40] showed that a large number of MCDM problems can be shown to be equivalent to a general formulation of LGP.

1.5 Contributions

Class imbalanced classification can also be thought of as a multi-criteria decision making problem, since the classification accuracy on the majority as well as the minority classes must be simultaneously maximized (these two objectives are often contradictory and cannot be maximized together, resulting in the need for a suitable trade-off). In spite of this, to the best of our knowledge, the current article is the first application of LxP to the class imbalanced classification problem.

Definition 3. We formally define a *Lexicographic Linear Program (LxLP)* as a lexicographic hierarchy of LPs (in the sense that the LPs in all the prior stages of the hierarchy must be solved before the LPs in the current stage can be solved). The j th LP to be solved at the i th stage is of the form

$$\begin{aligned} \mathbf{v}_{ij}^* &= \arg \min_{\mathbf{v}} L_{ij}(\mathbf{v}, \mathbf{v}_{i-1}^*), \\ \text{s. t. } g_k(\mathbf{v}, \mathbf{v}_{i-1}^*) &\leq 0 \quad \forall k \in \{1, \dots, \eta_i\} \\ \text{and } h_l(\mathbf{v}, \mathbf{v}_{i-1}^*) &= 0 \quad \forall l \in \{1, \dots, v_i\}, \end{aligned}$$

where η_i and v_i respectively are the number of inequality and equality constraints while L_{ij} is an appropriate loss function. The vector $\mathbf{v}_{i-1}^* = (\mathbf{v}_{(i-1)1}^*, \dots, \mathbf{v}_{(i-1)\zeta_{i-1}}^*, \dots, \mathbf{v}_{11}^*, \dots, \mathbf{v}_{1\zeta_1}^*)$

contains the optimal solutions to all LPs solved in all the

preceding stages with ξ_i denoting the number of LPs solved in the i th stage.

Based on the insights from Section 1.3, we propose a two staged scheme to choose the weights of the component classifiers of a boosted ensemble. The first stage is concerned with solving a set of LPs (one for each class, which can be solved in parallel) to find the two (possibly different) sets of component weights corresponding to the individual minimum attainable average hinge losses for the two classes. Subsequently, the second stage solves another LP to find the set of component weights that minimizes the maximum increase in the class-wise average hinge losses beyond the minimum values found in the first stage. It is clear that the proposed scheme is an LxLP problem as all LPs in the first stage must be solved before the final LP from the second stage can be formulated. The proposed method is referred to as LexiBoost hereafter. We also formulate a dual to LexiBoost, called Dual-LexiBoost, which not only selects optimal component weights but also adapts the point-wise weights, over the rounds of boosting, to counter the effects of class imbalance.

The proposed methods have the following advantages:

- It invokes the novel hinge loss based regularization method, which unlike the slack variable based regularization, does not require cost set tuning to achieve a good balance between the classes, thus addressing the long-standing issue of expensive cost set tuning for imbalanced data learning.
- Moreover, the proposed approach is readily applicable to multi-class or polychotomous learning tasks, which have as yet received limited attention in the class imbalanced learning literature.
- Even though we demonstrate the abilities of the proposal by using the AdaBoost algorithm, the proposed philosophy can be readily applied to other ensemble learning techniques (such as bagging [41]) as well.

1.6 Organization

We introduce the reader to some of the existing LP based boosting schemes in Section 2. We then provide a detailed explanation of the proposed two staged LxLP based LexiBoost framework in Section 3. The dual formulation resulting from the proposed LxLP is presented in Section 3.3. The proposed framework is also generalized to multi-class classification problems in Section 3.4. Computational complexity of the proposed methods is discussed in Section 3.5. Subsequently, experimental results are presented and discussed in Section 4. We conclude the article in Section 5.

2 LINEAR PROGRAMMING BASED BOOSTING

In this section, we introduce the reader to some of the extant LP based boosting techniques which are crucial to understanding the proposed improvement.

LPAdaBoost. Despite the theoretical guarantees on the training performance of AdaBoost [7], Grove and Schuurmans [28] proposed the LPAdaBoost algorithm to maximize the minimum margin ρ , aiming to achieve better generalization performance. The primal LP posed by LPAdaBoost is of the form

$$\begin{aligned} A_1: (\alpha^*, \rho^*) &= \arg \max \rho, \\ \text{s. t. } y_i \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i) &\geq \rho \quad \forall i \in \{1, \dots, n\}, \\ \sum_{t=1}^T \alpha_t &= 1, \\ \text{and } \alpha &\geq 0, \end{aligned}$$

where $f_t(\mathbf{x}_i)$ is the classifier generated by the t th round of AdaBoost.

Dual-LPAdaBoost. The dual to the primal LPAdaBoost formulation is

$$\begin{aligned} A'_1: (D_{t+1}^*, s^*) &= \arg \min s, \\ \text{s. t. } \sum_{i=1}^n D_{t+1}(i) y_i f'_t(\mathbf{x}_i) &\leq s \quad \forall t \in \{1, \dots, T\}, \\ \sum_{i=1}^n D_{t+1}(i) &= 1, \\ \text{and } D_{t+1} &\geq 0, \end{aligned}$$

where $f'_t(\mathbf{x}_i)$ is the classifier generated in the t th round. The dual formulation corresponds to assigning the point-wise weights $D_{t+1}(i)$ such that the aggregate margin s is minimized. To put it simply, the dual attempts to find a D_{t+1} which assigns the greatest weightage to the points which prove to be the most difficult during rounds 1 through t . The Dual-LPAdaBoost algorithm consists of alternately solving the LPs A_1 and A'_1 until the convergence criterion $s - \rho < 0$ is met or the maximum number of rounds T is reached.

LPBoost. Since the hard margin formulation of LPAdaBoost makes it sensitive to noise and outliers, Rätsch et al. [33] presented a soft margin variant called LPBoost which regularizes the outlier instances using slack variables ξ_i (corresponding to the data points $\mathbf{x}_i$), resulting in the following LP:

$$\begin{aligned} A_2: (\alpha^*, \xi^*, \rho^*) &= \arg \min -\rho + D \sum_{i=1}^n \xi_i, \\ \text{s. t. } y_i \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i) &\geq \rho - \xi_i \quad \forall i \in \{1, \dots, n\}, \\ \sum_{t=1}^T \alpha_t &= 1, \\ \alpha &\geq 0, \\ \text{and } \xi &\geq 0. \end{aligned}$$

An appropriately high cost D must be assigned for the regularization of data points in order to achieve good performance. This parameter has to be generally selected by cross-validation on the training data.

DualLPBoost. The dual LP arising out of the LPBoost formulation is of the form

$$\begin{aligned} A'_2: (D_{t+1}^*, s^*) &= \arg \min s, \\ \text{s. t. } \sum_{i=1}^n D_{t+1}(i) y_i f'_t(\mathbf{x}_i) &\leq s \quad \forall t \in \{1, \dots, T\}, \\ \sum_{i=1}^n D_{t+1}(i) &= 1, \\ \text{and } 0 &\leq D_{t+1} \leq D, \end{aligned}$$

giving rise to the Dual-LPBoost algorithm where A'_2 is solved for a maximum of T rounds (until the convergence criterion $\sum_{i=1}^n D_t(i) y_i f_t(\mathbf{x}_i) \leq s$ is satisfied) with the Lagrangian multipliers of A'_2 being chosen to be the component weights α_t .

LPUBoost. Leskovec and Shawe-Taylor [29] further adapted the LPBoost formulation to two-class imbalanced problems by introducing uneven costs for regularizing the two classes. The non-target (usually majority) class is assigned a regularization cost of D as in LPBoost, while the target (usually minority) class is assigned a higher regularization cost of βD ($\beta > 1$). The resulting primal LP is

$$\begin{aligned} A_3: (\alpha^*, \xi^*, \rho^*) = \arg \min \quad & -\rho + \beta D \sum_{i=1}^{n_1} \xi_i + D \sum_{i=n_1+1}^n \xi_i, \\ \text{s. t. } & y_i \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i) \geq \rho - \xi_i \quad \forall i \in \{1, \dots, n\}, \\ & \sum_{t=1}^T \alpha_t = 1, \\ & \text{and } (\alpha, \xi) \geq 0, \end{aligned}$$

where n_1 denotes the number of points in the positive (target) class. Hence, the number of points in the negative (non-target) class is $n_2 = n - n_1$. Both the parameters D as well as β must be selected by expensive tuning on $\mathbb{R}^+ \times \mathbb{R}^+$ using cross-validation. Tuning the parameter β essentially corresponds to tuning the relative cost between the two classes, and is critical to achieving good performance.

Dual-LPUBoost. Like LPAdaBoost and LPBoost, LPUBoost also gives rise to a dual problem. The dual problem is of the form

$$\begin{aligned} A'_3: (D_{t+1}^*, s^*) = \arg \min \quad & s, \\ \text{s. t. } & \sum_{i=1}^n D_{t+1}(i) y_i f_t(\mathbf{x}_i) \leq s \quad \forall t \in \{1, \dots, T\}, \\ & \sum_{i=1}^n D_{t+1}(i) = 1, \\ & 0 \leq D_{t+1}(i) \leq \beta D \quad \forall i \in \{1, \dots, n_1\}, \\ & \text{and } 0 \leq D_{t+1}(i) \leq D \quad \forall i \in \{n_1 + 1, \dots, n\}. \end{aligned} \quad (1)$$

However, the Dual-LPUBoost algorithm solves a slightly modified form of the LP A'_3 to accommodate for the drawbacks of simple cost set tuning. While the modification does seem to lend some robustness to the method (see Section 4), it also adds an additional tunable parameter D_{LB} , which determines the lower limit of $D_{t+1}(i)$ as a fraction of the corresponding upper limit. Thus, the changes pertain to the constraints (1) and (2), resulting in the modified constraints

$$\begin{aligned} \beta D \times D_{LB} \leq D_{t+1}(i) \leq \beta D \quad \forall i \in \{1, \dots, n_1\}, \\ \text{and } D \times D_{LB} \leq D_{t+1}(i) \leq D \quad \forall i \in \{n_1 + 1, \dots, n\}. \end{aligned}$$

The termination criterion and the choice of component weights are identical to those of Dual-LPBoost.

3 LEXICOGRAPHIC LINEAR PROGRAMMING BASED SELECTION OF COMPONENT WEIGHTS

Having acquainted the reader to the existing LP based boosting schemes, we now elucidate the proposed Lexi-Boost algorithm which uses a two staged LxLP. The two stages of LP involved in our proposed LxLP framework are formally defined in Sections 3.1 and 3.2.

3.1 The First Stage of LPs

As already mentioned in Section 1.5, the initial aim is to choose the component classifier weights so that the average hinge loss on the differences between the actual and ideal margins is minimized for the individual classes. Since the hinge loss is piece-wise linear, the minimization problems can be posed as LPs. Therefore, an LP P_j (j denotes the class in question) of the following form must be solved for each of the classes:

$$P_j: (\alpha_j, \lambda_j^*) = \arg \min \quad \frac{1}{n_j} \sum_{i=1}^{n_j} \lambda_i, \quad (3)$$

$$\text{s. t. } 1 - \rho(\mathbf{x}_i) \leq \lambda_i \quad \forall i \in \{1, \dots, n_j\},$$

$$\sum_{t=1}^T \alpha_t = 1, \quad (4)$$

$$\alpha \geq 0, \quad (5)$$

$$\text{and } \lambda \geq 0, \quad (6)$$

where n_j is the number of points in the j th class ($j \in \{1, \dots, |\mathcal{C}|\}$, $\mathcal{C} = \{c_1, c_2\}$ for the two-class imbalanced problem being the set of classes), λ_i are the slack variables which measure the hinge loss for the points $\mathbf{x}_i$ by serving as the upper bound on the difference between the ideal and actual margin values (which depends on the choice of the component weights α_t as per Definition 1). λ_j^* denotes the optimal set of λ_i values obtained by solving the LP P_j , and α_j is the corresponding set of component weights.

3.2 The Final LP

Since the outlier instances already have high loss values even when the overall class-wise losses are minimized, any further increase in the class-wise losses is likely to be due to regularization of non-outlier instances. Therefore, having obtained the optimal hinge losses λ_j^* for each of the individual classes, the deviations from average optimal losses must be minimized to restrict the regularization (and hence misclassification) of non-outlier points. Hence, we finally solve another LP, referred to as Q, to find the α_t values which minimize χ the maximum of such deviations as per constraint (7). χ^* denotes the optimized value of χ . It should be noted that unlike P_j , all the classes (and hence all the training data points) are considered together at this stage. The LP Q is run to obtain the optimal set of component weights α^* which strike a good balance between the classes. The formulation² for the program is as follows:

2. If a known cost c_j is attached to the class c_j in an application, constraint (7) should be modified to $\frac{c_j}{n_j} \sum_{i \in c_j} \lambda_i - \frac{c_j}{n_j} \mathbf{e}^T \lambda_j^* \leq \chi \quad \forall j \in \{1, \dots, |\mathcal{C}|\}$.

$$\begin{aligned} Q: (\alpha^*, \lambda, \chi^*) &= \arg \min \chi, \\ \text{s. t. } \frac{1}{n_j} \sum_{i \in c_j} \lambda_i - \frac{1}{n_j} \mathbf{e}^T \lambda_j^* &\leq \chi \quad \forall j \in \{1, \dots, |\mathcal{C}|\}, \end{aligned} \quad (7)$$

$$1 - \rho(\mathbf{x}_i) \leq \lambda_i \quad \forall i \in \{1, \dots, n\}, \quad (8)$$

$$\sum_{t=1}^T \alpha_t = 1, \quad (9)$$

$$\alpha \geq 0, \quad (10)$$

$$\lambda \geq 0, \quad (11)$$

$$\text{and } \chi \geq 0, \quad (12)$$

where λ_i denotes the new loss (possibly different from the loss $\lambda_{ji}^* \in \lambda_j^*$ obtained from P_j) for the point $\mathbf{x}_i \in c_j$, α_t are the corresponding component weights, and $\mathbf{e}$ denotes a vector of ones, having appropriate length. Hence, the complete LexiBoost algorithm (in its primal formulation) is presented in Algorithm 1.

Algorithm 1. LexiBoost

Input: Dataset X of labelled instances $\mathbf{x}_i$.

Output: Final ensemble classifier

$$H(\mathbf{x}_i) = \text{sign}(\sum_{t=1}^T \alpha_t^* f_t(\mathbf{x}_i)).$$

Run AdaBoost on the dataset X to obtain $f_t(\mathbf{x}_i)$.

for all $j \in \{1, \dots, |\mathcal{C}|\}$ **do**

 Solve LP P_j to obtain λ_j^* .

end for

Solve LP Q to obtain α^* .

3.3 The Dual to LexiBoost

The dual to boosting techniques are often of interest. This is principally because the dual formulations allow not only for suitable choice of the component weights but also facilitate suitable choices of the point-specific weights over the rounds. Moreover, from a theoretical point of view, the dual formulations can be used to unify seemingly different boosting techniques such as AdaBoost, LPBoost, etc. [42]. Therefore, in this section we present the dual formulation of the proposed LexiBoost algorithm.

The Lagrangians arising from the LPs P_j of the LexiBoost primal are of the form

$$\begin{aligned} \mathcal{L}_j(\alpha, \lambda, D, \sigma, \gamma, s) &= \frac{1}{n_j} \sum_{i=1}^{n_j} \lambda_i + s \left(\sum_{t=1}^T \alpha_t - 1 \right) \\ &+ \sum_{i=1}^{n_j} D(i) (1 - \lambda_i - y_i \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i)) - \sum_{t=1}^T \sigma_t \alpha_t - \sum_{i=1}^{n_j} \gamma_i \lambda_i, \end{aligned}$$

where $D(i)$, s , σ_t , and γ_i are respectively the Lagrangian multipliers corresponding to the constraints (3), (4), (5), and (6). Differentiating $\mathcal{L}_j$ w. r. t. λ_i and α_t and equating to zero, we get the constraints

$$\frac{\partial \mathcal{L}_j}{\partial \lambda_i} = \frac{1}{n_j} - D(i) - \gamma_i = 0, \quad (13)$$

$$\text{and } \frac{\partial \mathcal{L}_j}{\partial \alpha_t} = s - \sum_{i=1}^{n_j} D(i) y_i f_t(\mathbf{x}_i) - \sigma_t = 0. \quad (14)$$

Imposing constraints (13) and (14) in the Lagrangian, and eliminating σ_t and γ_i from the constraints, we get the dual LPs of the form

$$\begin{aligned} &\text{maximize } \sum_{i=1}^{n_j} D(i) - s, \\ \text{s. t. } &\sum_{i=1}^n D(i) y_i f_t(\mathbf{x}_i) \leq s, \\ &0 \leq D(i) \leq \frac{1}{n_j} \quad \forall i \in c_j. \end{aligned} \quad (15)$$

Now, since the dual LPs for boosting algorithms can be used for choosing suitable point-specific weights subsequent to each round of boosting, we slightly modify the LP in (15) to obtain the dual LPs P'_j of the form

$$\begin{aligned} P'_j: (D_{t+1}^*, s^*) &= \arg \max \sum_{i=1}^{n_j} D_{t+1}(i) - s, \\ \text{s. t. } &\sum_{i=1}^n D_{t+1}(i) y_i f_\tau(\mathbf{x}_i) \leq s \quad \forall \tau \in \{1, \dots, t\}, \\ &\sum_{i=1}^{n_j} D_{t+1}(i) = 1, \end{aligned} \quad (16)$$

$$0 \leq D_{t+1}(i) \leq \frac{1}{n_j} \quad \forall i \in c_j, \forall j \in \{1, \dots, |\mathcal{C}|\}, \quad (17)$$

where $D_{t+1}(i)$ (corresponding to the $D(i)$ in (15)) is the point-specific weight of the data point $\mathbf{x}_i$. We have imposed the additional constraint (16) to ensure that the $D_{t+1}(i)$ values can be meaningfully used as data point weights. It is easy to see that the LP remains feasible despite the introduction of constraint (16) as the case where $D_{t+1}(i) = \frac{1}{|\mathcal{C}|n_j} \forall i \in c_j \forall j \in \{1, \dots, |\mathcal{C}|\}$ is a feasible solution. However, the introduction of the constraints means that a sub-optimal solution for (15) may be the optimal solution for P'_j , resulting in the optimal objective function value of P'_j being lower than that of P_j (despite there being no duality gap between (15) and P_j , as strong duality holds for LP problems).

The interpretation of P'_j is similar to that of LPAdaBoost in that the dual formulation attempts to assign higher weights to the points which have proved to be difficult in the previous rounds 1 through t while also maximizing the sum of weights. The constraints (17) ensure that higher weights are assigned to instances belonging to the minority class.

Additionally, the Lagrangian corresponding to the final LP Q of the LexiBoost primal is as follows:

$$\begin{aligned} \mathcal{L}(\alpha, \lambda, \chi, d, D, \sigma, \gamma, s, \psi) = & \chi + s \left(\sum_{t=1}^T \alpha_t - 1 \right) \\ & \sum_{j=1}^{|C|} d_j \left(\frac{1}{n_j} \sum_{i \in c_j} \lambda_i - \frac{1}{n_j} \sum_{i \in c_j} \lambda_{ji}^* - \chi \right) \\ & + \sum_{i=1}^n D(i) \left(1 - \lambda_i - y_i \sum_{t=1}^T \alpha_t f_t(\mathbf{x}_i) \right) \\ & - \sum_{t=1}^T \sigma_t \alpha_t - \sum_{i=1}^n \gamma_i \lambda_i - \psi \chi, \end{aligned}$$

where d_j , $D(i)$, s , σ_t , γ_i , and ψ are respectively the Lagrangian multipliers corresponding to the constraints (7), (8), (9), (10), (11), and (12). Differentiating $\mathcal{L}$ w. r. t. χ , λ_i , and α_t and equating to zero, we get the constraints

$$\frac{\partial \mathcal{L}}{\partial \chi} = 1 - \sum_{j=1}^{|C|} d_j - \psi = 0, \quad (18)$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_i} = \frac{d_j}{n_j} - D(i) - \gamma_i = 0, \quad (19)$$

$$\text{and } \frac{\partial \mathcal{L}}{\partial \alpha_t} = s - \sum_{i=1}^n D(i) y_i f_t(\mathbf{x}_i) - \sigma_t = 0. \quad (20)$$

Imposing constraints (18), (19), and (20) in the Lagrangian, and eliminating σ_t , γ_i , and ψ from the constraints, we get the dual LPs of the form

$$\begin{aligned} \max. & \sum_{i=1}^n D(i) - \sum_{j=1}^{|C|} \frac{d_j}{n_j} \mathbf{e}^T \lambda_j^* - s, \\ \text{s. t. } & \sum_{i=1}^n D(i) y_i f_t(\mathbf{x}_i) \leq s, \\ & 0 \leq D(i) \leq \frac{d_j}{n_j} \quad \forall i \in c_j, \forall j \in \{1, \dots, |C|\}, \\ & \sum_{j=1}^{|C|} d_j \leq 1, \quad d_j \geq 0, \end{aligned}$$

which we modify in a manner similar to the LPs P'_j to obtain the following LP Q' :

$$\begin{aligned} Q': (D_{t+1}^*, d, s^*) = & \arg \max \sum_{i=1}^n D_{t+1}(i) - \sum_{j=1}^{|C|} \frac{d_j}{n_j} \mathbf{e}^T \lambda_j^* - s, \\ \text{s. t. } & \sum_{i=1}^n D_{t+1}(i) y_i f_\tau(\mathbf{x}_i) \leq s \quad \forall \tau \in \{1, \dots, t\}, \\ & \sum_{i=1}^n D_{t+1}(i) = 1, \\ & 0 \leq D_{t+1}(i) \leq \frac{d_j}{n_j} \quad \forall i \in c_j, \forall j \in \{1, \dots, |C|\}, \\ & \sum_{j=1}^{|C|} d_j \leq 1, \\ & d \geq 0. \end{aligned} \quad (21)$$

It is easy to see that the introduction of the constraint (21) does not make Q' infeasible as ensuring $D_{t+1}(i) = \frac{d_j}{n_j} \forall i \in c_j$

$\forall j \in \{1, \dots, |C|\}$ for some d such that $\sum_{j=1}^{|C|} d_j = 1$ is enough to yield feasible solutions. However, similar to P'_j , the introduction of the additional constraint may result in the optimal objective function value of Q' being lower than that of Q .

It can be seen that the LP Q' is similar to the LPs P'_j and only differs in that the upper bounds of the instance weights $D_{t+1}(i)$ are scaled by an amount d_j (for $\mathbf{x}_i \in c_j$) which is inversely proportional to the average hinge loss obtained for the corresponding class in the first stage of LPs. In other words, greater regularization is induced (by enforcing a lower upper bound for instance weights) for the class having a greater proportion of outlier instances, while also maintaining higher weightage for the non-outlier minority class instances to compensate for class imbalance.

Based on these two dual formulations, we now present the complete Dual-LexiBoost method as Algorithm 2.

Algorithm 2. Dual-LexiBoost

Input: Dataset X of labelled instances $\mathbf{x}_i$.

Output: Final ensemble classifier

$$H(\mathbf{x}_i) = \text{sign} \left(\sum_{t=1}^T \alpha_t^* f_t(\mathbf{x}_i) \right).$$

Initialize $D_1(i) = 1/(|C|n_j) \quad \forall i \in c_j, \forall j \in \{1, \dots, |C|\}$.

for all $t = 1$ to T **do**

Train weak classifier f_t using distribution D_t .

Calculate training error ϵ_t using distribution D_t .

if $\epsilon_t > \frac{1}{|C|}$ **then break**

end if

for all $j \in \{1, \dots, |C|\}$ **do**

Solve LP P'_j to obtain $D_{t+1}(i) \quad \forall i \in c_j$.

end for

end for

for all $j \in C$ **do**

Solve LP P_j to obtain λ_j^* .

end for

Initialize $D_1(i) = 1/(|C|n_j) \quad \forall i \in c_j, \forall j \in \{1, \dots, |C|\}$.

for all $t = 1$ to T **do**

Train weak classifier f_t using distribution D_t .

Calculate training error ϵ_t using distribution D_t .

if $\epsilon_t > \frac{1}{|C|}$ **then break end if**

Solve LP Q' to obtain $D_{t+1}(i) \quad \forall i \in \{1, \dots, n\}$.

end for

Solve LP Q to obtain α^* .

3.4 Generalization to Multi-Class Tasks

It is easy to see that the proposed approach is readily applicable to polychotomous or multi-class classification tasks. However, the definition of margin must be altered for the multi-class setting in the following way:

Definition 4. Let $X = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ be a given multi-class training dataset with label vectors $\mathbf{y}_i \in \mathbb{R}^{|C|}$ such that

$$y_{i,j} = \begin{cases} 1 & \text{if } \mathbf{x}_i \text{ belongs to the } j\text{th class,} \\ -1 & \text{otherwise.} \end{cases}$$

Additionally, let $\mathbf{f}_t(\mathbf{x}_i)$ denote the prediction vector for the point $\mathbf{x}_i$ by the classifier f_t . Then the margin $\rho(\mathbf{x}_i)$ can be redefined as

TABLE 1
Time Complexities of
Linear Programming Based Boosting Variants

Algorithm	Primal	Dual
LPAdaBoost	$O(n)$	$O(n^5)$
LPBoost	$O(C_D n^5)$	$O(C_D n^5)$
LPUBoost	$O(C_\beta C_D n^5)$	$O(C_\beta C_D C_{LB} n^5)$
LexiBoost	$O(n^5)$	$O(n^5)$

C_D : No. of candidate values of the parameter D for LPBoost and Dual-LPBoost.

C_β : No. of candidate values of the parameter β for LPUBoost and Dual-LPUBoost.

C_{LB} : No. of candidate values of the parameter D_{LB} for Dual-LPUBoost.

$$\rho(\mathbf{x}_i) = \sum_{t=1}^T \alpha_t \mathbf{y}_i^T \mathbf{f}_t(\mathbf{x}_i).$$

Thereafter, LexiBoost as well as Dual-LexiBoost can be directly applied to multi-class problems by modifying the LPs P_j , Q , P'_j , and Q' accordingly.

3.5 Time Complexity of LexiBoost

It is well-known that the time complexity of interior-point methods for solving LPs is $O(\mathfrak{N}^3 \mathcal{L})$, where $\mathfrak{N}$ is the number of variables and $\mathcal{L}$ is the size of the input data, i.e., the number of bits required to encode the coefficients of the objective function and the constraints of the LPs [43]. Now, the LPs P_j of the LexiBoost primal have $(n_j + T)$ variables and requires $(2n_j + T + 1)(n_j + T + 1) + T$ coefficients for characterizing the objective function and the constraints. Assuming T to be a constant and assuming that all the coefficients are encoded using a fixed number of bits, the time complexity of the LPs P_j becomes $O(\sum_{j=1}^{|C|} n_j^5)$. The final LP Q of the LexiBoost primal has $(n + T + 1)$ variables and is characterized by $(|C| + 2n + T + 2)(n + T + 2) + T$ coefficients. Making the additional assumption that the number of classes is much lower than the number of data points (i.e., $n \gg |C|$), the time complexity for Q becomes $O(n^5)$. Thus, the total complexity of

the LexiBoost primal LPs becomes $O(n^5)$. On the other hand, the LPs P'_j for Dual-LexiBoost have $(n + 1)$ variables and require $2(n + 1) + (t + n)(n + 2)$ coefficients, resulting in a complexity of $O(n^5)$. The final LP Q' for Dual-LexiBoost has $(n + |C| + 1)$ variables and is characterized by $(2n + |C| + 2) + (t + n)(n + |C| + 1)$ coefficients, resulting in a time complexity of $O(n^5)$. Hence, the LPs for Dual-LexiBoost have a total time complexity of $O(n^5)$. We compare the asymptotic order of time complexities of the methods discussed in Section 2 against those of LexiBoost and Dual-LexiBoost in Table 1. It can be observed from Table 1 that both LexiBoost and Dual-LexiBoost enjoy lower complexity compared to the primal as well as dual variants of LPBoost and LPUBoost as the proposed methods do not require cost tuning. For a comparison of the actual training times, see Fig. 3.

4 EXPERIMENTS

In this section, we report the results of experiments conducted on two-class artificial datasets of varying specifications, two-class as well as multi-class real-world datasets, multi-class hyperspectral image classification, and multi-class classification of a class imbalanced subset of the ImageNet dataset. Our implementation of LexiBoost can be found at <https://github.com/Shounak-D/LexiBoost>.

4.1 Competitors and Experimental Setup

We compare our results with AdaBoost (the AdaBoost.M2 variant [44] being used for multi-class datasets) which serves as a baseline, with AdaMEC-Calib which has recently been found to be quite effective for imbalanced datasets [6], and with the negative correlation based AdaBoost.NC in conjunction with random oversampling which has been shown to be effective on imbalanced data [20]. We also compare our results with those of the primal solutions (i.e., using classifiers already created by AdaBoost) as well as dual solutions to LPAdaBoost (because of its inherent ability to tackle imbalance in noise and outlier-free situations) and LPUBoost. The C4.5 decision tree [45] and the k -Nearest

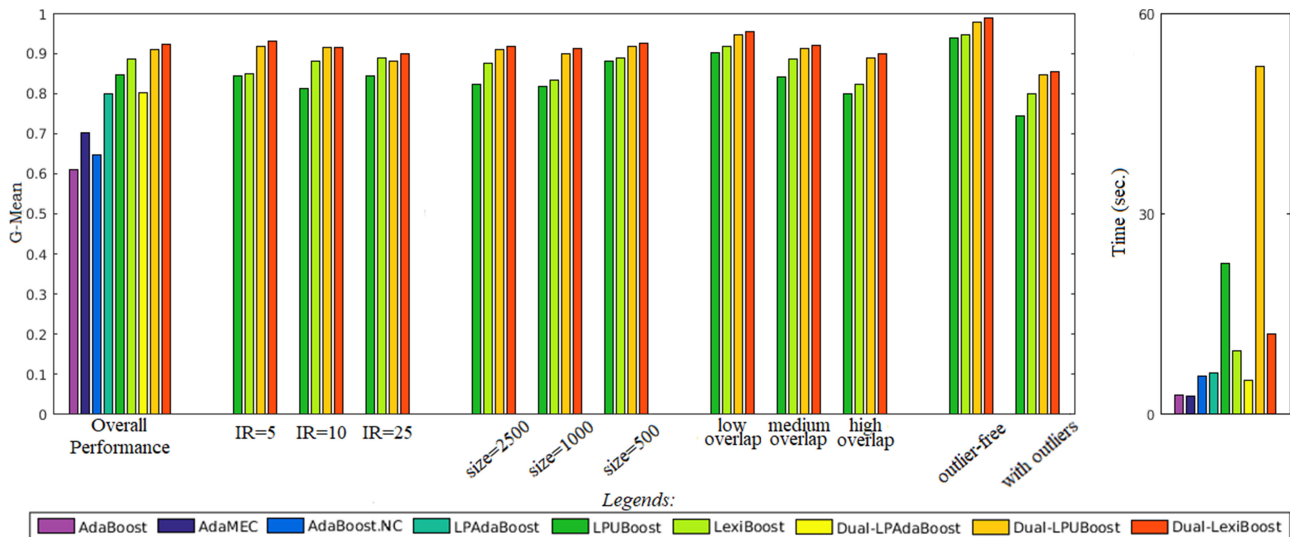


Fig. 3. Average G-Mean values for k -NN on artificial datasets: LexiBoost and Dual-LexiBoost are compared with the two best contenders.

TABLE 2
Parameter Settings for Contending Algorithms

Algorithm	Parameter settings	Experiments
Baseline methods :		
AdaBoost [7]	$T = 10$	Two-class
AdaBoost.M2 [44]	$T = 10$	Multi-class
AdaMEC-Calib [17]	$T = 10$; $^1C_+ = \text{IR}$; $C_- = 1$; Two random samplings of 80 percent of the training data are used for training and calibration	Two-class Multi-class
AdaBoost.NC [20]	$T = 10$; $^2\lambda_{NC} \in \{2, 9\}$ as per [20]; Random oversampling is done so as to equate the number of points in all classes	Two-class Multi-class
LP based primal methods :		
LPAdaBoost [28]	$T = 10$	Two-class, Multi-class
LPUBoost [29]	$T = 10$; $v \in \{0.1, 0.2\}$; $D = \frac{1}{v}$; $\beta \in \{2, 4, 8\}$	Two-class
LP based dual methods :		
Dual-LPAdaBoost [28]	$T = 10$; $^3\epsilon_{LPA} = 1 \times 10^{-16}$	Two-class, Multi-class
Dual-LPUBoost [29]	$T = 10$; $v \in \{0.1, 0.2\}$; $D = \frac{1}{v}$; $\beta \in \{2, 4, 8\}$; $D_{LB} \in \{25, 50, 100\}$	Two-class
Proposed methods :		
LexiBoost	$T = 10$	Two-class, Multi-class
Dual-LexiBoost	$T = 10$	Two-class, Multi-class

¹ C_+ and C_- denote the costs of false negatives and false positives, respectively.

² λ_{NC} controls the strength of the penalty term in AdaBoost.NC.

³ ϵ_{LPA} is the tolerance for primal-dual convergence in Dual-LPAdaBoost.

Neighbor (k NN) classifier are used as base learners. The experiments on the real-world datasets are reported with both C4.5 and k NN as base classifiers. Only the results with k NN as the base classifier are reported for the experiments on the artificial datasets, hyperspectral images and ImageNet, as the k NN based variants are observed to generally perform better in these cases.³ The parameters for C4.5 are chosen as per [45] while the parameter k for k NN is varied in the range $\{3, 5, 10\}$ for all sets of experiments except for hyperspectral image classification, where $k = 10$ is used because of the relatively large size of the datasets. The parameter settings used for evaluating each of the contenders are summarized in Table 2. The contending methods which are directly extendable to multi-class cases are used for the multi-class experiments, as indicated in Table 2.

The performance for the experiments is reported using the G-Mean [46], AUC [47], and Avg-AUC [48] indexes. The G-Mean measure is calculated as the geometric mean of the individual class-wise accuracies. Since the G-Mean index has a high value only when the performance is good on all classes, it is useful for evaluating the performance for class imbalanced classification. The AUC index, on the other hand, measures the expected proportion of positive data samples which are more likely to be assigned to the positive class, compared to a randomly drawn negative sample. Since the AUC measure is only defined for two-class classification tasks, Hand and Till [48] proposed the Avg-AUC index as an extension of the AUC measure to multi-class problems using one-versus-one decomposition (i.e., average AUC over all possible pairings of the individual classes). Formal definitions of the indexes can be found in the supplementary

document, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TKDE.2019.2894148>. The results are presented in the following sections using the average index values corresponding to the best parameter settings for each contending algorithm.

4.2 Artificial Datasets

We create 27 two-class artificial datasets by sampling points from two distinct 5-dimensional standard normal distributions by varying the Imbalance Ratio (IR) in $\{5, 10, 25\}$ and the total size of the datasets in $\{500, 1000, 2500\}$. The overlap between the classes is also varied by keeping the centre for the minority class fixed at $[0, 0, 0, 0, 0]^T$ while the centre for the majority class is varied between $[3, 3, 3, 3, 3]^T$, $[1.7, 1.7, 1.7, 1.7, 1.7]^T$, and $[1.5, 1.5, 1.5, 1.5, 1.5]^T$. 27 analogous datasets with outliers are also created by replacing 10 percent of the instances of each class with instances from the opposite class. The G-Mean values obtained for these 54 datasets using k NN as base classifier are summarized from different perspectives in Fig. 3. LPUBoost and Dual-LPUBoost, respectively being the best primal and dual methods apart from the proposed methods, are used for the comparison over varying IR, size, overlap and presence of outliers.

Fig. 3 indicates that LexiBoost consistently performs better than the other primal techniques as well as the baselines AdaBoost, AdaMEC-Calib, and AdaBoost.NC. Similarly, Dual-LexiBoost consistently performs best among all the algorithms. In fact, Dual-LexiBoost even outperforms the exhaustive cost tuning methods, namely LPUBoost and Dual-LPUBoost. Moreover, the time required for training LexiBoost and Dual-LexiBoost is much lower than that of their corresponding closest rivals LPUBoost and Dual-LPUBoost, and is comparable to that of the baseline techniques like AdaBoost.

This points towards the effectiveness of the proposed

3. The corresponding results using C4.5 can be found in the supplementary document, available online.

TABLE 3
Summary of Results on Real-World Datasets

				AdaBoost	AdaMEC-Calib	AdaBoost_NC	LPAdaBoost	LPUBoost	LexiBoost	Dual-LPAdaBoost	Dual-LPUBoost	Dual-LexiBoost
Two-class datasets	C4.5 as base classifier :											
	AUC	Average Rank		6.80	4.90	8.07	5.30	4.27	2.70	7.23	3.17	2.57
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_1	H_1	H_1	H_1	H_1	-	H_1	H_0	H_0
			CN: Dual-LexiBoost	H_1	H_1	H_1	H_1	H_0	H_0	H_1	H_0	-
	G-Mean	Average Rank		6.97	4.53	8.30	5.40	4.27	2.77	7.23	3.03	2.50
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_1	H_1	H_1	H_1	H_1	-	H_1	H_0	H_0
			CN: Dual-LexiBoost	H_1	H_1	H_1	H_1	H_0	H_0	H_1	H_0	-
	kNN as base classifier :											
	AUC	Average Rank		7.80	6.00	6.97	6.10	4.73	3.10	5.27	2.63	2.40
		Friedman Test		H_1								
WSRT		CN: LexiBoost	H_1	H_1	H_1	H_1	H_1	-	H_1	H_0	H_0	
		CN: Dual-LexiBoost	H_1	H_1	H_1	H_1	H_1	H_0	H_1	H_0	-	
G-Mean	Average Rank		7.87	5.67	6.83	6.37	4.53	2.93	5.77	2.83	2.20	
	Friedman Test		H_1									
	WSRT	CN: LexiBoost	H_1	H_1	H_1	H_1	H_1	-	H_1	H_0	H_0	
		CN: Dual-LexiBoost	H_1	H_1	H_1	H_1	H_1	H_0	H_1	H_1	-	
Multi-class datasets	C4.5 as base classifier :											
	Avg-AUC	Average Rank		3.70	N/A	3.80	5.10	N/A	1.80	4.70	N/A	1.90
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_1	N/A	H_1	H_1	N/A	-	H_1	N/A	H_0
			CN: Dual-LexiBoost	H_1	N/A	H_1	H_1	N/A	H_0	H_1	N/A	-
	G-Mean	Average Rank		3.75	N/A	3.80	4.80	N/A	2.55	4.00	N/A	2.10
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_0	N/A	H_0	H_1	N/A	-	H_1	N/A	H_0
			CN: Dual-LexiBoost	H_1	N/A	H_1	H_1	N/A	H_0	H_1	N/A	-
	kNN as base classifier :											
	Avg-AUC	Average Rank		4.40	N/A	3.95	4.60	N/A	2.50	4.30	N/A	1.25
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_1	N/A	H_0	H_1	N/A	-	H_1	N/A	H_1
			CN: Dual-LexiBoost	H_1	N/A	H_1	H_1	N/A	H_1	H_1	N/A	-
	G-Mean	Average Rank		4.10	N/A	4.25	4.75	N/A	2.70	3.60	N/A	1.60
		Friedman Test		H_1								
		WSRT	CN: LexiBoost	H_1	N/A	H_1	H_1	N/A	-	H_0	N/A	H_1
			CN: Dual-LexiBoost	H_1	N/A	H_1	H_1	N/A	H_1	H_1	N/A	-

H_0 : Contenders perform similarly WSRT : Wilcoxon Signed Rank Test.

H_1 : Significant difference among contenders CN : Control method.

Best values shown in **boldface** N/A : Not used for multi-class experiments.

framework for finding the best trade-off between classes, without cost tuning. An empirical validation of the ability of the LexiBoost framework to circumvent the requirement for cost tuning can be found in the supplementary material, available online. It is important to note here that combining post-calibration with boundary-shifting enables AdaMEC-Calib to perform much better than AdaBoost, despite requiring the lowest training time among all the contenders. However, it does not perform as well as the other imbalance handling schemes, LPUBoost and Dual-LPUBoost. This is possibly because the final ensemble classifier learned by AdaBoost may be miscalibrated in a way which cannot be compensated for by shifting the decision threshold. Instead, carefully choosing the component weights may be able to help in such cases, as indicated by the relatively better performance of LPUBoost.

4.3 Real-World Datasets

For the experiments on real-world imbalanced datasets, we use 15 two-class and 10 multi-class datasets with varying degrees of imbalance from the KEEL repository [49] (see

supplementary document for details, available online). All the methods are compared on the two-class datasets while only AdaBoost (the AdaBoost.M2 variant), LPAdaBoost, LexiBoost, Dual-LPAdaBoost and Dual-LexiBoost are compared on the multi-class datasets (as the other methods are not directly adaptable to multi-class problems).

4.3.1 Two-Class Classification

The performance over the 15 two-class real-world imbalanced datasets is summarized in Table 3 for C4.5 and k NN as the base classifier, respectively. The results are summarized in terms of average ranks, Friedman test hypotheses, and Wilcoxon signed rank test hypotheses [50]. The Friedman test [51] is used to ascertain whether there is significant difference among the performances of the various contenders. Since the Friedman test suggests significant difference in all cases, we further employ the signed rank test to investigate the pair-wise differences between the contenders. It is seen that Dual-LexiBoost achieves the best rank, followed by LexiBoost and Dual-LPUBoost. The performances of Dual-LexiBoost,

LexiBoost and Dual-LPUBoost are found to be statistically equivalent in terms of both G-Mean and AUC while all the other contenders are found to perform significantly worse than both Dual-LexiBoost and LexiBoost for both choices of base classifiers. This indicates that the proposed framework has the capacity to perform at least as good as exhaustive cost tuning methods, even on real-world datasets. The lower average rankings of Dual-LexiBoost (relative to LexiBoost) indicate that the dual formulation for adapting instance weights is indeed useful for achieving better performance.

4.3.2 Multi-Class Classification

The performance of AdaBoost.M2, AdaBoost.NC, LPAdaBoost, LexiBoost, Dual-LPAdaBoost and Dual-LexiBoost (on the multi-class real-world datasets having varying number of classes and degree of imbalance) is summarized for C4.5 and k NN respectively in the lower half of Table 3. The Friedman test detects significant difference among the contenders in all cases. Hence, the Wilcoxon signed rank test, along with average ranks are used for summarizing the performance. The overall performance is the best for Dual-LexiBoost followed by LexiBoost. The performance of Dual-LexiBoost is significantly better than that of all other methods in terms of both indexes. This indicates that the dual method of instance weight adaptation is useful for generating better balance between the classes for multi-class datasets as well.

4.4 Hyperspectral Image Classification

Hyperspectral image classification has been listed, in a recent survey by Krawczyk [52], as one of the key practical application areas where multi-class imbalance naturally arises. Therefore, in this section, we test the effectiveness of the proposed techniques for this application. For the experiments on hyperspectral image classification, we use the Samson and Jasper ridge images from [53], the Kennedy Space Center (KSC) image from [54], and the Salinas A and Indian pines scenes from [55]. For the images which contain unlabeled pixels, the training and testing is only undertaken on the labeled pixels as per [56]. We only use k NN (with $k = 10$) as the base classifier for these experiments because of the large size of these datasets. The results are summarized in Table 4 in terms of the average ranking and the rank-sum test [50], [57] win, tie, loss counts for both Avg-AUC and G-Mean. The Kruskal-Wallis test [58] is used to detect difference among all contenders for each dataset. The wins, ties, and losses (of the contender against the control, viz. LexiBoost or Dual-LexiBoost) are only counted on the datasets which have significant differences among the contenders according to the Kruskal-Wallis test. We use the Kruskal-Wallis and rank-sum tests for each dataset, instead of using the Friedman and signed rank tests across datasets, owing to the limited number of datasets. The effectiveness of the proposed methods is attested to by their low average ranks and is also visible from the illustration in Fig. 4 for the KSC image.

4.5 ImageNet Classification

One of the more challenging applications of pattern recognition is the classification of natural images. Uncurated natural image datasets are inherently class imbalanced. Moreover,

TABLE 4
Summary of Results on Hyperspectral Images and ImageNet

			AdaBoost	AdaBoost_NC	LPAdaBoost	LexiBoost	Dual-LPAdaBoost	Dual-LexiBoost
Hyperspectral Images :								
Avg-AUC	Avg. Rank		3.40	4.60	5.50	2.40	4.10	1.00
	CN: Lexi-Boost	W	0	0	0	-	0	1
		T	2	1	1	-	2	1
		L	0	1	1	-	0	0
	CN: Dual-LexiBoost	W	0	0	0	0	0	-
		T	1	0	0	1	1	-
L		1	2	2	1	1	-	
G-Mean	Avg. Rank		3.40	4.70	5.50	2.40	4.00	1.00
	CN: Lexi-Boost	W	0	0	0	-	0	1
		T	2	0	1	-	1	1
		L	0	2	1	-	1	0
	CN: Dual-LexiBoost	W	0	0	0	0	0	-
		T	1	0	0	1	1	-
L		1	2	2	1	1	-	
ImageNet :								
Avg-AUC	Avg. Rank		3.67	3.00	5.67	2.67	5.00	1.00
	CN: Lexi-Boost	W	0	0	0	-	0	1
		T	3	3	2	-	2	2
		L	0	0	1	-	1	0
	CN: Dual-LexiBoost	W	0	0	0	0	0	-
		T	0	1	0	1	0	-
L		3	2	3	2	3	-	
G-Mean	Avg. Rank		4.33	3.00	5.00	3.00	4.67	1.00
	CN: Lexi-Boost	W	0	0	0	-	0	3
		T	2	3	2	-	1	0
		L	1	0	1	-	2	0
	CN: Dual-LexiBoost	W	0	0	0	0	0	-
		T	0	0	0	0	0	-
L		3	3	3	3	3	-	

W, T, L: Wilcoxon rank-sum test Win, Tie, Loss counts, resp.
Best values shown in **boldface** CN : Control method.

the class distributions for such datasets are generally complex, making learning algorithms more sensitive to issues such as cost set tuning and outlier regularization. Therefore, in this section, we undertake the classification of imbalanced subsets of the popular ImageNet dataset [59]. We prepare 3 datasets, namely ImageNet8, ImageNet9, and ImageNet12 for this purpose. The datasets are prepared by randomly choosing images corresponding to the 8 principal subtrees of the ImageNet dataset as well as the miscellaneous subtrees, viz. Foods, Collections, Documents, and Microorganisms, from the ImageNet 2011 Fall Release. The number of images collected from each of the subtrees corresponds to about 2 percent of the number of synsets contained within the subtree in question, with the constraint that at least 20 images must be chosen from each subtree. The data sampling hierarchy thus obtained is illustrated in Fig. 5. The dataset ImageNet8 is prepared by only combining the samples from the 8 principal subtrees and not including the miscellaneous images, giving rise to a dataset containing 1120 images. The ImageNet9 dataset adds to the complexity of the classification task by appending the 120 miscellaneous images as a single class, resulting in a dataset of size 1240. The complexity is increased further in the ImageNet12 dataset as the images belonging to the miscellaneous subtrees Foods, Collections, Documents, and Microorganisms are classified

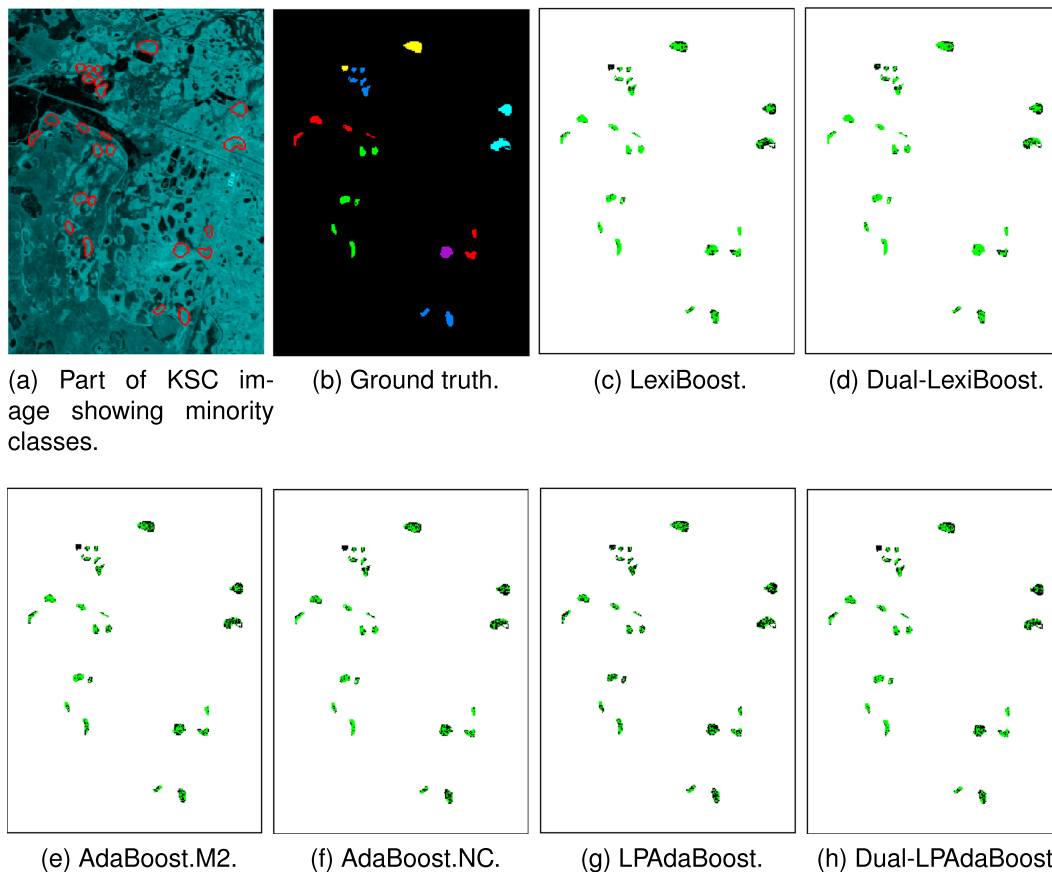


Fig. 4. Comparison of the contending methods showing better performance of the proposal on minority classes. Misclassified points are shown in black and correctly-classified points are shown in green.

into 4 different classes corresponding to these subtrees. In keeping with the state-of-the-art in feature representation of images, we derive a 2048-dimensional deep feature space representation of each image from the final global average pooling layer of the Inception-v3 deep neural network [60]. The results achieved by AdaBoost.M2, AdaBoost.NC, LPAdaBoost, LexiBoost, Dual-LPAdaBoost and Dual-LexiBoost are also summarized in Table 4. Yet again, Dual-LexiBoost is observed to achieve the best rank followed by LexiBoost. The fact that Dual-LexiBoost (unlike LexiBoost) exhibits tie counts of zero against all other contenders in terms of G-Mean indicates that the dual formulation can generate proper instance weights to improve the performance on all classes (as opposed to only some of the classes by LexiBoost) for multi-class datasets.

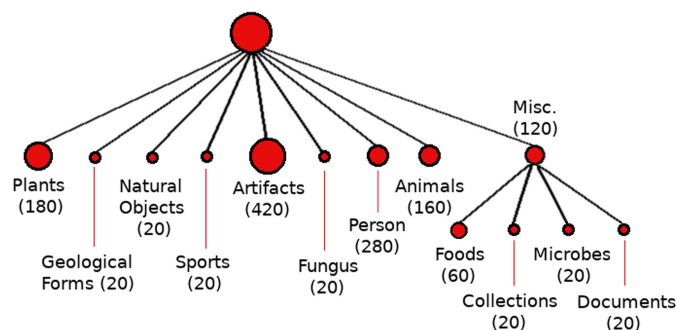


Fig. 5. ImageNet subset hierarchy.

Authorized licensed use limited to: Tsinghua University. Downloaded on November 17, 2025 at 06:07:33 UTC from IEEE Xplore. Restrictions apply.

5 CONCLUSIONS

Based on the understanding that the choice of component classifier weights for boosting can be thought of as a game of *Tug of War* between the classes in the margin space, we introduce the reader to a two-staged LxLP framework for handling class imbalance. The proposed framework, called LexiBoost, introduces a novel regularization scheme which offers an advantages over the traditional slack-variable-reliant scheme, due to the fact that the proposed scheme does not require to undertake expensive cost set tuning which has been the norm for imbalanced classification till date. Hence, the proposed framework also facilitates easy extension to multi-class problems. This makes LexiBoost directly applicable to both two-class as well as multi-class tasks. We also derive the dual algorithm corresponding to the proposed method, called Dual-LexiBoost. Experiments conducted on artificial datasets, real-world imbalanced datasets and hyperspectral images suggest that the proposed methods exhibit greater immunity to class imbalance, overlap, size of the dataset, as well as the presence of outliers. Dual-LexiBoost, owing to its ability to generate suitable point-specific weights, generally performs better than the primal method. In the near future, the authors plan to extend the proposed framework to single-class classification along the lines of [61].

ACKNOWLEDGMENTS

The authors would like to thank Anubhav Agrawal, a final year student pursuing a BTech degree in electronics and

electrical engineering from the Indian Institute of Technology, Guwahati, India, for helping with the computer implementation of some of the methods used in our experiments. This paper has supplementary downloadable material for online publication only, as provided by the authors. This includes details of the experimental results summarized in this article.

REFERENCES

- [1] R. E. Schapire, "The strength of weak learnability," *Mach. Learn.*, vol. 5, no. 2, pp. 197–227, 1990.
- [2] R. E. Schapire, Y. Freund, P. Bartlett, W. S. Lee, et al., "Boosting the margin: A new explanation for the effectiveness of voting methods," *Ann. Statistics*, vol. 26, no. 5, pp. 1651–1686, 1998.
- [3] M. A. Mazurowski, P. A. Habas, J. M. Zurada, J. Y. Lo, J. A. Baker, and G. D. Tourassi, "Training neural network classifiers for medical decision making: The effects of imbalanced datasets on classification performance," *Neural Netw.*, vol. 21, no. 2, pp. 427–436, 2008.
- [4] C. Phua, D. Alahakoon, and V. Lee, "Minority report in fraud detection: Classification of skewed data," *ACM Sigkdd Explorations Newsletter*, vol. 6, no. 1, pp. 50–59, 2004.
- [5] C. Huang, Y. Li, C. C. Loy, and X. Tang, "Learning deep representation for imbalanced classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 5375–5384.
- [6] N. Nikolaou, N. Edakunni, M. Kull, P. Flach, and G. Brown, "Cost-sensitive boosting algorithms: Do we really need them?" *Mach. Learn.*, vol. 104, no. 2–3, pp. 359–384, 2016.
- [7] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," in *Proc. Eur. Conf. Comput. Learn. Theory*, 1995, pp. 23–37.
- [8] K. M. Ting, "A comparative study of cost-sensitive boosting algorithms," in *Proc. 17th Int. Conf. Mach. Learn.*, 2000, pp. 983–990.
- [9] P. Viola and M. Jones, "Fast and robust classification using asymmetric Adaboost and a detector cascade," in *Proc. Advances Neural Inf. Process. Syst.*, pp. 1311–1318, vol. 2, 2002.
- [10] Y. Sun, M. S. Kamel, A. K. Wong, and Y. Wang, "Cost-sensitive boosting for classification of imbalanced data," *Pattern Recognit.*, vol. 40, no. 12, pp. 3358–3378, 2007.
- [11] H. Masnadi-Shirazi and N. Vasconcelos, "Asymmetric boosting," in *Proc. 24th Int. Conf. Mach. Learn.*, 2007, pp. 609–619.
- [12] H. Masnadi-Shirazi and N. Vasconcelos, "Cost-sensitive boosting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 2, pp. 294–309, Feb. 2011.
- [13] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Trans. Syst. Man Cybern. Part C (Appl. Rev.)*, vol. 42, no. 4, pp. 463–484, Jul. 2012.
- [14] I. Landesa-Vázquez and J. L. Alba-Castro, "Double-base asymmetric Adaboost," *Neurocomputing*, vol. 118, pp. 101–114, 2013.
- [15] B. Wang and J. Pineau, "Online bagging and boosting for imbalanced data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 12, pp. 3353–3366, Dec. 2016.
- [16] M. Ohsaki, P. Wang, K. Matsuda, S. Katagiri, H. Watanabe, and A. Ralescu, "Confusion-matrix-based kernel logistic regression for imbalanced data classification," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 9, pp. 1806–1819, Sep. 2017.
- [17] N. Nikolaou and G. Brown, "Calibrating Adaboost for asymmetric learning," in *Proc. Int. Workshop Multiple Classifier Syst.*, 2015, pp. 112–124.
- [18] Z.-H. Zhou and X.-Y. Liu, "On multi-class cost-sensitive learning," *Comput. Intell.*, vol. 26, no. 3, pp. 232–257, 2010.
- [19] B. Krawczyk, "Cost-sensitive one-vs-one ensemble for multi-class imbalanced data," in *Proc. Int. Joint Conf. Neural Netw.*, 2016, pp. 2447–2452.
- [20] S. Wang and X. Yao, "Multiclass imbalance problems: Analysis and potential solutions," *IEEE Trans. Syst. Man Cybern. Part B (Cybern.)*, vol. 42, no. 4, pp. 1119–1130, Aug. 2012.
- [21] N. V. Chawla, A. Lazarevic, L. O. Hall, and K. W. Bowyer, "SMOTEBoost: Improving prediction of the minority class in boosting," in *Proc. 7th Eur. Conf. Principles Practice Knowl. Discovery Databases*, 2003, pp. 107–119.
- [22] S. Chen, H. He, and E. A. Garcia, "RAMOBoost: Ranked minority oversampling in boosting," *IEEE Trans. Neural Netw.*, vol. 21, no. 10, pp. 1624–1642, Oct. 2010.
- [23] H. Guo and H. L. Viktor, "Learning from imbalanced data sets with boosting and data generation: The DataBoost-IM approach," *ACM Sigkdd Explorations Newslett.*, vol. 6, no. 1, pp. 30–39, 2004.
- [24] X. Yang, Q. Kuang, W. Zhang, and G. Zhang, "AMDO: An over-sampling technique for multi-class imbalanced problems," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1672–1685, Sep. 2018.
- [25] D. Mease, A. Wyner, and A. Buja, "Cost-weighted boosting with jittering and over/under-sampling: JOUS-Boost," *J. Mach. Learn. Res.*, vol. 8, pp. 409–439, 2007.
- [26] C. Seiffert, T. M. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "RUSBoost: A hybrid approach to alleviating class imbalance," *IEEE Trans. Syst. Man Cybern.-Part A: Syst. Humans*, vol. 40, no. 1, pp. 185–197, Jan. 2010.
- [27] M. V. Joshi, V. Kumar, and R. C. Agarwal, "Evaluating boosting algorithms to classify rare classes: Comparison and improvements," in *Proc. IEEE Int. Conf. Data Mining*, 2001, pp. 257–264.
- [28] A. J. Grove and D. Schuurmans, "Boosting in the limit: Maximizing the margin of learned ensembles," in *Proc. 15th Nat./10th Conf. Artif. Intell./Innovative Appl. Artif. Intell.*, 1998, pp. 692–699.
- [29] J. Leskovec and J. Shawe-Taylor, "Linear programming boosting for uneven datasets," in *Proc. 20th Int. Conf. Mach. Learn.*, 2003, pp. 456–463.
- [30] S. Das, S. Datta, and B. B. Chaudhuri, "Handling data irregularities in classification: Foundations, trends, and future challenges," *Pattern Recognit.*, vol. 81, pp. 674–693, 2018.
- [31] F. Charro, J. G. Azorero, and J. D. Rossi, "A mixed problem for the infinity Laplacian via tug-of-war games," *Calculus Variations Partial Differential Equations*, vol. 34, no. 3, pp. 307–320, 2009.
- [32] Y. Peres, O. Schramm, S. Sheffield, and D. B. Wilson, "Tug-of-war and the infinity Laplacian," in *Selected Works of Oded Schramm*. Berlin, Germany: Springer, 2011, pp. 595–638.
- [33] G. Rätsch, T. Onoda, and K.-R. Müller, "Soft margins for Adaboost," *Mach. Learn.*, vol. 42, no. 3, pp. 287–320, 2001.
- [34] J. P. Ignizio, *Goal Programming and Extensions*. Lanham, MD, USA: Lexington Books, 1976.
- [35] J. Sankaran, "On a variant of lexicographic multi-objective programming," *Eur. J. Oper. Res.*, vol. 107, no. 3, pp. 669–674, 1998.
- [36] Y. Ijiri, *Management Goals and Accounting for Control*. Amsterdam, Netherlands: North Holland Pub. Co., 1965, vol. 3.
- [37] S. M. Lee, et al., *Goal Programming for Decision Analysis*. Philadelphia, PA, USA: Auerbach Publishers, 1972.
- [38] L. Pourkarimi and M. Zarepisheh, "A dual-based algorithm for solving lexicographic multiple objective programs," *Eur. J. Oper. Res.*, vol. 176, no. 3, pp. 1348–1356, 2007.
- [39] M. Cococcioni, M. Pappalardo, and Y. D. Sergeyev, "Lexicographic multi-objective linear programming using grossone methodology: Theory and algorithm," *Appl. Math. Comput.*, vol. 318, pp. 298–311, 2018.
- [40] C. Romero, "Extended lexicographic goal programming: A unifying approach," *Omega*, vol. 29, no. 1, pp. 63–71, 2001.
- [41] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.
- [42] C. Shen and H. Li, "On the dual formulation of boosting algorithms," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 12, pp. 2216–2231, Dec. 2010.
- [43] F. A. Potra and S. J. Wright, "Interior-point methods," *J. Comput. Appl. Math.*, vol. 124, pp. 281–302, 2000.
- [44] Y. Freund, R. E. Schapire, et al., "Experiments with a new boosting algorithm," in *Proc. 13th Int. Conf. Mach. Learn.*, 1996, vol. 96, pp. 148–156.
- [45] J. R. Quinlan, *C4. 5: Programs for Machine Learning*. Amsterdam, The Netherlands: Elsevier, 2014.
- [46] M. Kubat, S. Matwin, et al., "Addressing the curse of imbalanced training sets: One-sided selection," in *Proc. Int. Conf. Mach. Learn.*, 1997, vol. 97, pp. 179–186.
- [47] M. A. Maloof, "Learning when data sets are imbalanced and when costs are unequal and unknown," in *Proc. Workshop Learn. Imbalanced Data Sets II*, 2003, vol. 2, pp. 2–1.
- [48] D. J. Hand and R. J. Till, "A simple generalisation of the area under the ROC curve for multiple class classification problems," *Mach. Learn.*, vol. 45, no. 2, pp. 171–186, 2001.
- [49] I. Triguero, S. González, J. M. Moyano, S. García, J. Alcalá-Fdez, J. Luengo, A. Fernández, M. J. del Jesus, L. Sánchez, and F. Herrera, "KEEL 3.0: An open source software for multi-stage analysis in data mining," *Int. J. Comput. Intell. Syst.*, vol. 10, pp. 1238–1249, 2017, [Online]. Available: <http://sci2s.ugr.es/keel/datasets.php>.

- [50] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [51] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," *J. American Statistical Assoc.*, vol. 32, no. 200, pp. 675–701, 1937.
- [52] B. Krawczyk, "Learning from imbalanced data: open challenges and future directions," *Progress Artif. Intell.*, vol. 5, no. 4, pp. 221–232, 2016.
- [53] F. Zhu, Y. Wang, B. Fan, S. Xiang, G. Meng, and C. Pan, "Spectral unmixing via data-guided sparsity," *IEEE Trans. Image Process.*, vol. 23, no. 12, pp. 5412–5427, Dec. 2014.
- [54] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.
- [55] "Hyperspectral remote sensing scenes," [Online]. Available: http://www.ehu.es/ccwintco/index.php?title=Hyperspectral_Remote_Sensing_Scenes, Accessed on: Aug. 24, 2017.
- [56] T. Sun, L. Jiao, J. Feng, F. Liu, and X. Zhang, "Imbalanced hyperspectral image classification based on maximum margin," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 3, pp. 522–526, Mar. 2015.
- [57] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statistics*, vol. 18, pp. 50–60, 1947.
- [58] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *J. Amer. Statistical Assoc.*, vol. 47, no. 260, pp. 583–621, 1952.
- [59] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [60] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
- [61] G. Rätsch, B. Schölkopf, S. Mika, and K.-R. Müller, "Constructing boosting algorithms from SVMs: An application to one-class classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 9, pp. 1184–1199, Sep. 2002.



Shounak Datta received the BTech degree in electronics and communication engineering from the West Bengal University of Technology, Kolkata, India in 2011, and the ME degree in electronics and telecommunication engineering from Jadavpur University, Kolkata, India, in 2013. He is currently working toward the PhD degree in computer science at the Indian Statistical Institute, Kolkata, India. His research interests include imbalanced classification, learning with missing features, multi-objective optimization in machine learning, etc.



Sayak Nag received the BE degree in instrumentation and electronics engineering from Jadavpur University, Kolkata, India. His research interests include ensembles classifiers, support vector machines, neural networks, multi-objective optimization, and machine learning in general.



Swagatam Das is currently serving as an associate professor with the Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, India. He has published more than 250 research articles in peer-reviewed journals and international conferences. He is the founding co-editor-in-chief of *Swarm and Evolutionary Computation*, an international journal from Elsevier. He has more than 16,000 Google Scholar citations and an H-index of 60 to date.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.